

UML State-Chart/ State-Machine Diagrams

UML Statechart Diagram (STD)

- A Statechart diagram describes a state machine.
- State machine can be defined as a machine which defines different states of an object and these states are controlled by external or internal events.
- Shows the sequences of states that objects of a class go through during its life cycle in response to external events and also the responses and actions in reaction to an event.
- Activity Diagrams are special type of state diagrams where activities are activity states and state is changed upon the completion of an activity

Purpose

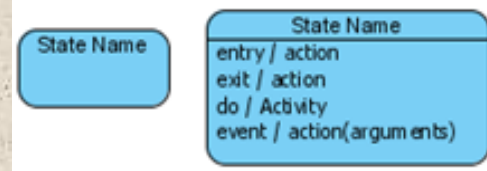
- Statechart diagram is one of the five UML diagrams used to model dynamic nature of a system.
- They define different states of an object during its lifetime. And these states are changed by events.
- Statechart diagrams are useful to model reactive systems. Reactive systems can be defined as a system that responds to external or internal events.
- The most important purpose of Statechart diagram is to model life time of an object from creation to termination.

STD Elements

- **A State** - is the condition of an object at a moment in time
 - the time between events.

– e.g. a phone is in the *state of being “idle”* after the receiver is placed on the hook until it is taken off the hook.

– Initial State: ●

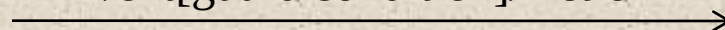


Final State:



- **A transition** - is a relationship between two states that indicates that when an event occurs, the object moves from the prior state to the subsequent state.
 - e.g. when the the event “off hook” occurs, *transition the phone from the “idle” to “active” state.*

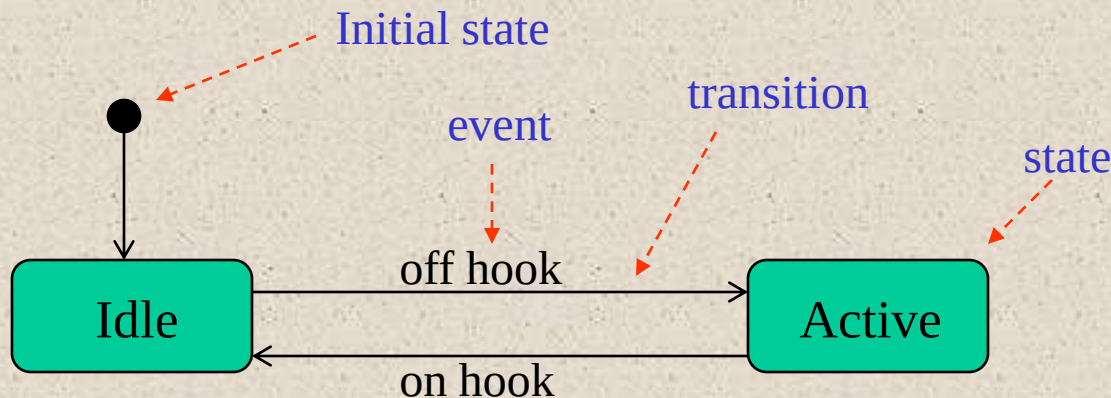
Event[guard condition]/Action



STD Elements

- **A Event** - is a significant or noteworthy occurrence at a point in time.
 - e.g. a phone receiver is taken off the hook

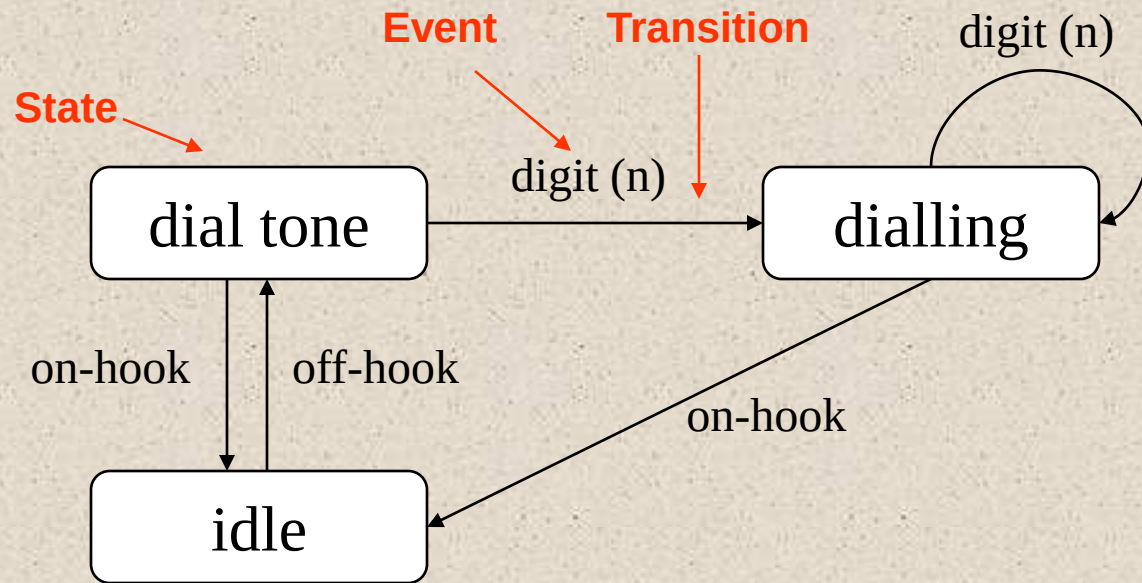
Example: STD for a Phone



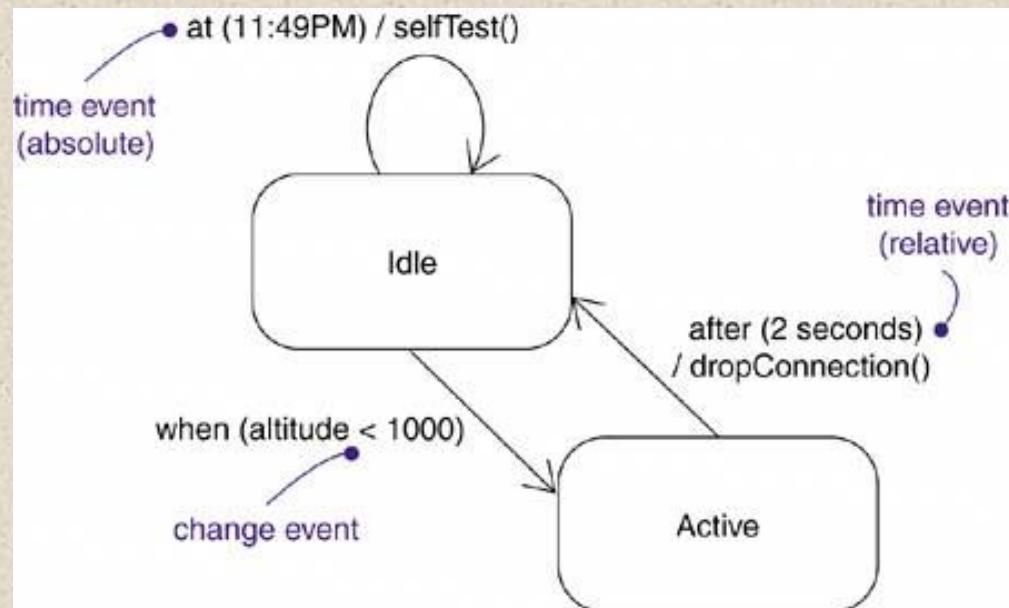
Event Types

- Signal Event
 - is event of sending or receiving signal
 - It is an explicit one way transmission of information from one object to another object
 - e.g. when a user dials a digit OR enters a string, an signal event has occurred.
- Change Event
 - is caused by satisfaction of a boolean expression.
 - Shown as **when (boolean expression)**
 - e.g. when(number<3) OR when(battery power<limit)
- Temporal Event
 - is caused by the occurrence of a specific date and time or passage of time.
 - E.g. when(date = jan 1,2010) OR after(10 s) OR at(11:45 p.m.)

Signal Event



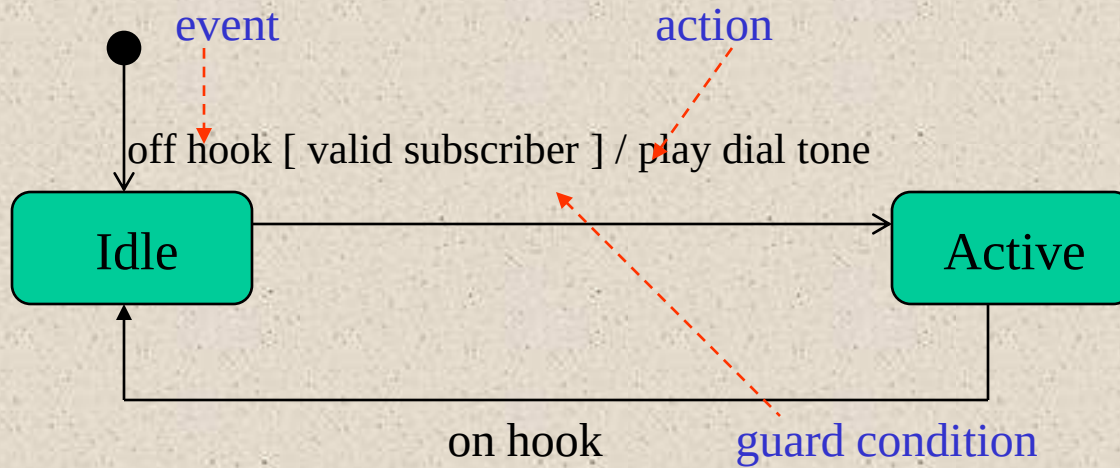
Time & Change Event



Transition Actions, Guard Conditions

- Transition Actions
 - a transition can cause an action to fire. In SW, this may represent the invocation of a method of an object
- Transition Guard conditions
 - a transition may also have a conditional guard -- or boolean test. The transition is only taken if the test passes.

Transition Action and Guards



Guard Condition

- Boolean function of object values
- Valid over an interval of time
- Can be used as guards on transitions
- Guard condition shown in brackets, following event name

Operations

- Attached to states or transitions
- Performed in response to corresponding states or events
- Types
 - Activity
 - Action

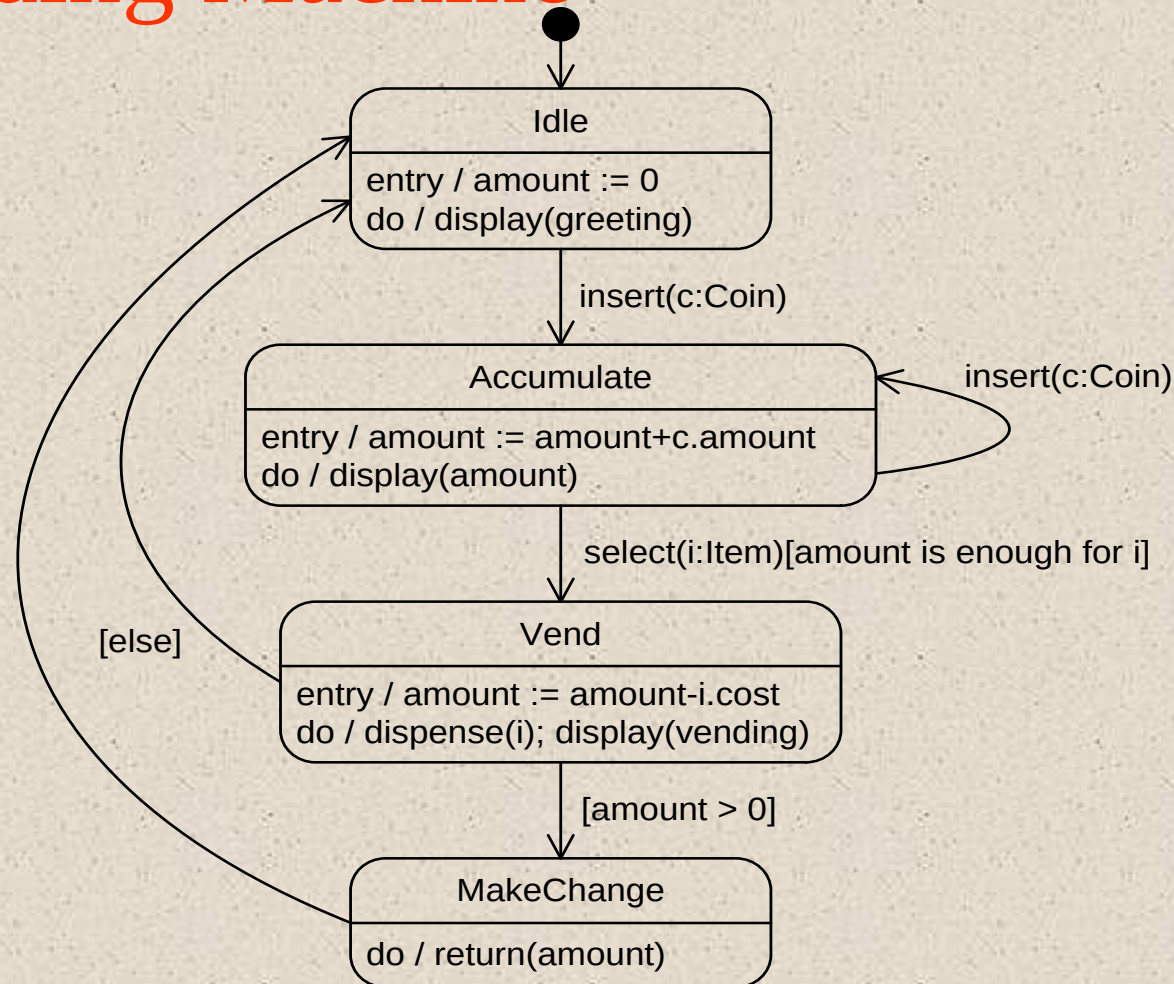
Operations

- Activity
 - operation that takes time to complete
 - associated with a state
 - include continuous or sequential operations
 - Some other outgoing transition may result in:
 - The interruption of the activity, and
 - An early exit from the state.
 - notation “do/ A” within a state box
 - indicates activity A
 - starts on entry
 - ends on exit

Activities

Activity Label	Meaning
Entry	Execute the associated <i>action-expression</i> (an <i>entry action</i>) upon state entry.
Exit	Execute the associated <i>action-expression</i> (an <i>exit action</i>) upon state exit.
Do	Execute the associated <i>action-expression</i> (a <i>do activity</i>) upon state entry and continue until state exit or action completion.

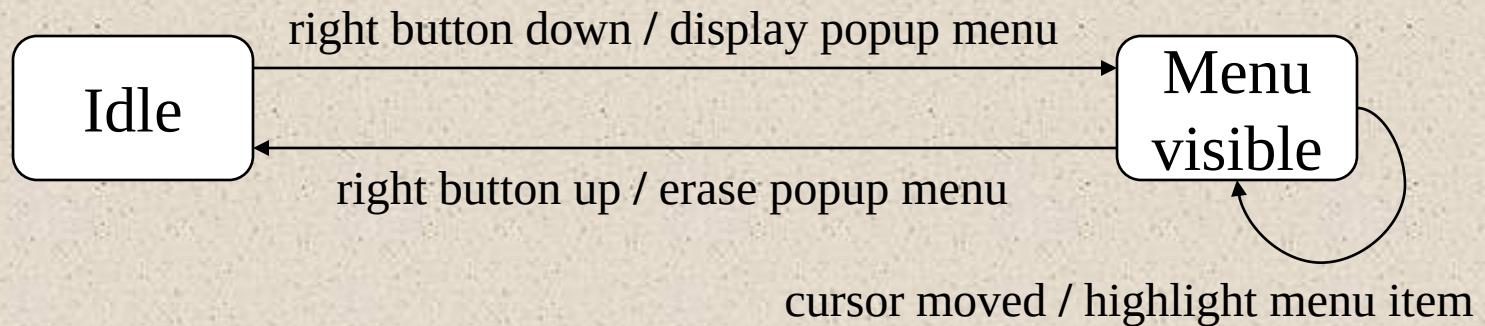
Example - State Activities for Vending Machine



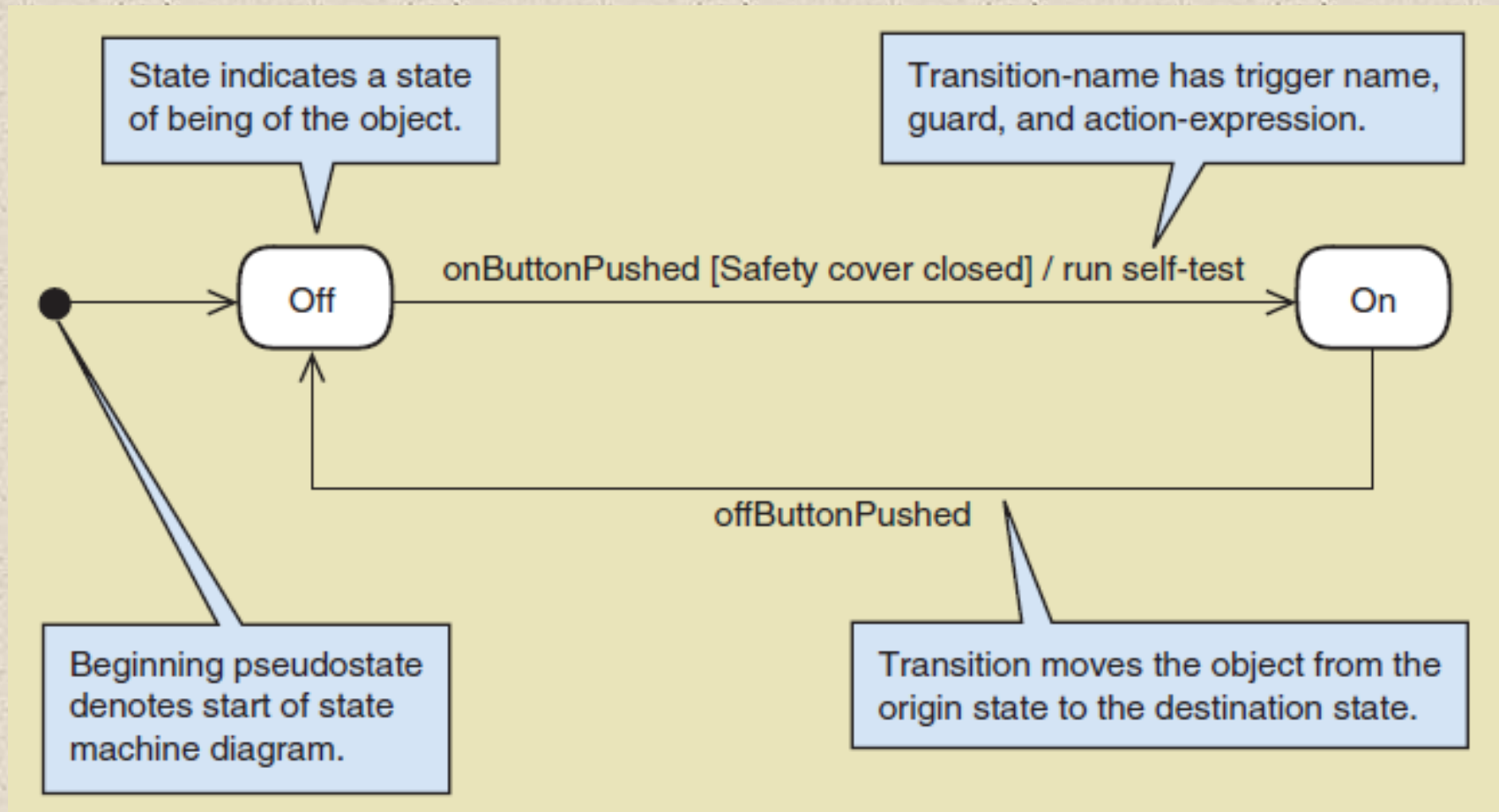
Operations

- Action
 - instantaneous operation
 - associated with an event
 - notation
 - slash (“/”) and name of the action, following the event

Example - Transition Actions

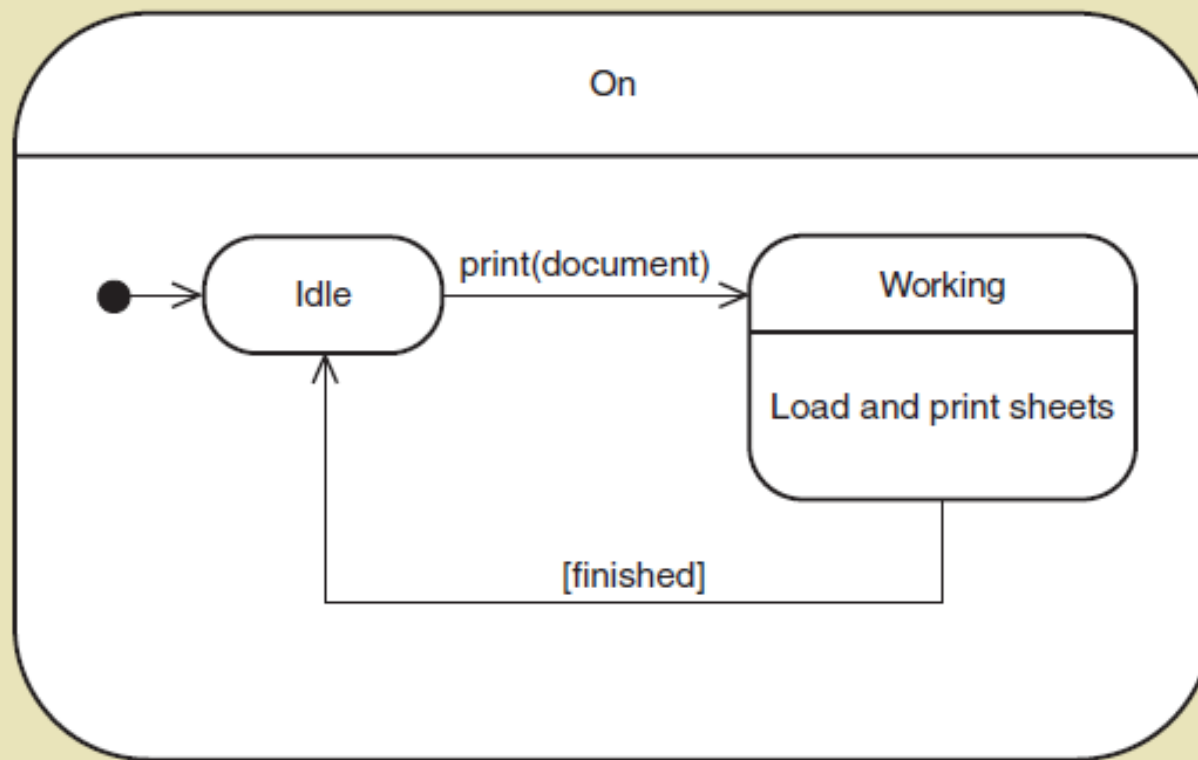


State Machine Diagram for a Printer



Nested State Diagrams

- A state without a nested state compartment is a **simple state**; one with a nested state compartment is a **composite state**



Composite States

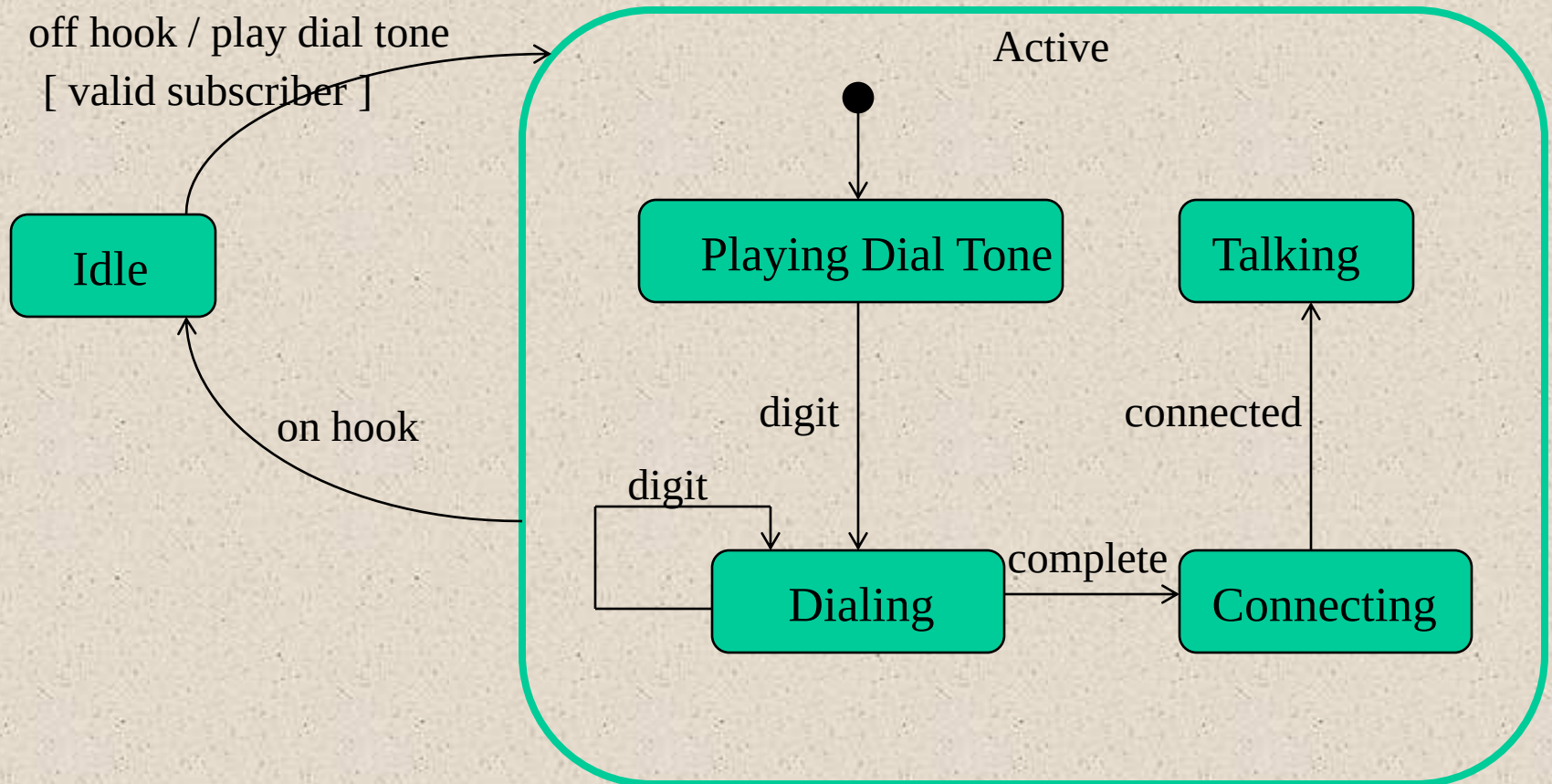
- ***Sequential composite state***—The nested state compartment is a single region with *sub-states* or *inner states* and transitions.
- ***Concurrent composite state***—The nested state compartment is comprised of two or more regions separated by *concurrent region boundary lines* (dashed lines) containing inner states and transitions.

Sequential composite state

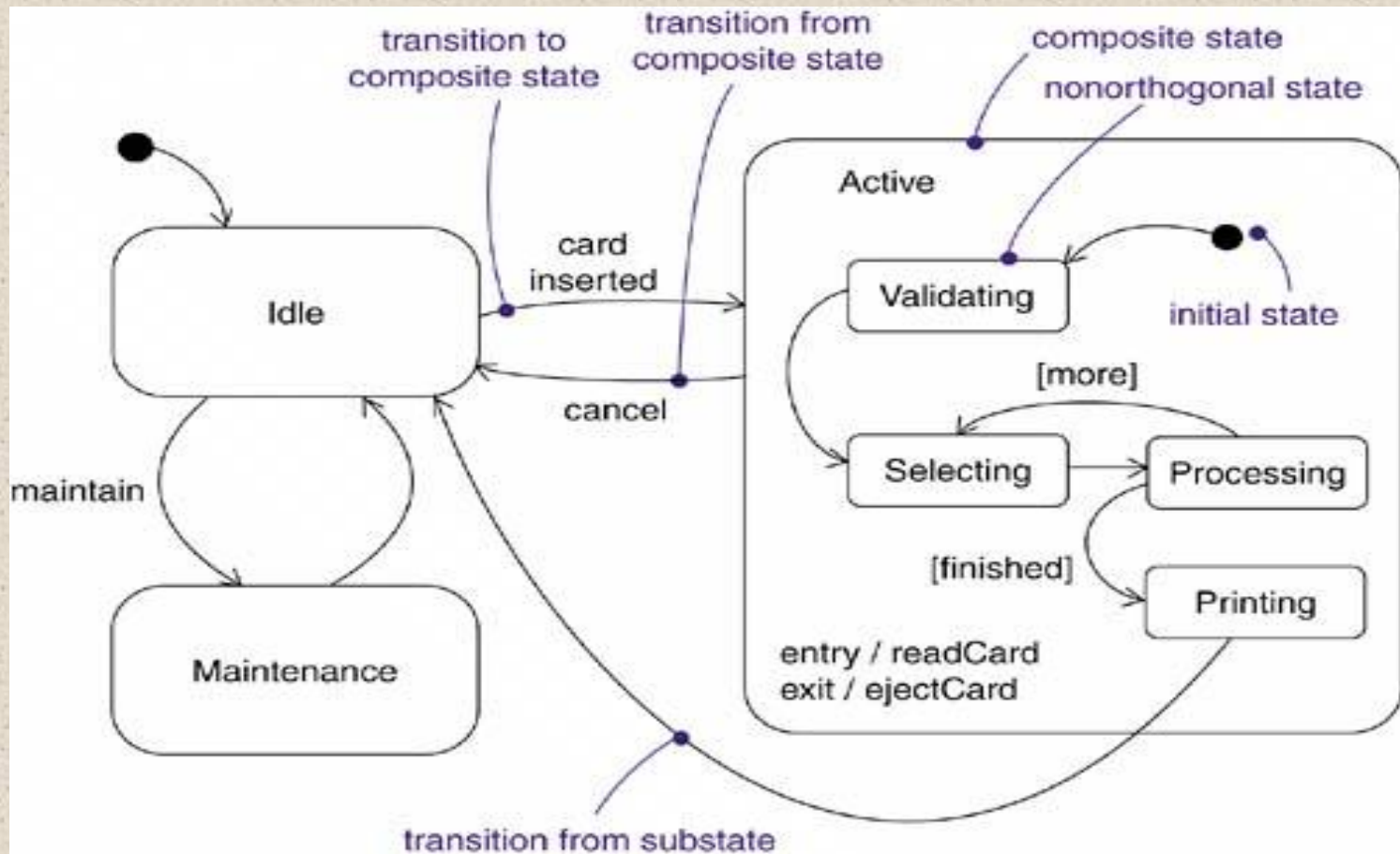
- When a sequential composite state is entered, so is one of its inner states (and likewise for further nested states), and they jointly become the current state.
 - Nested diagrams must have initial states or transitions must go directly to inner states
- When a sequential composite state is exited, so is the current inner state (and likewise for further nested states).

Example: Sequential Nested States

Phone Line

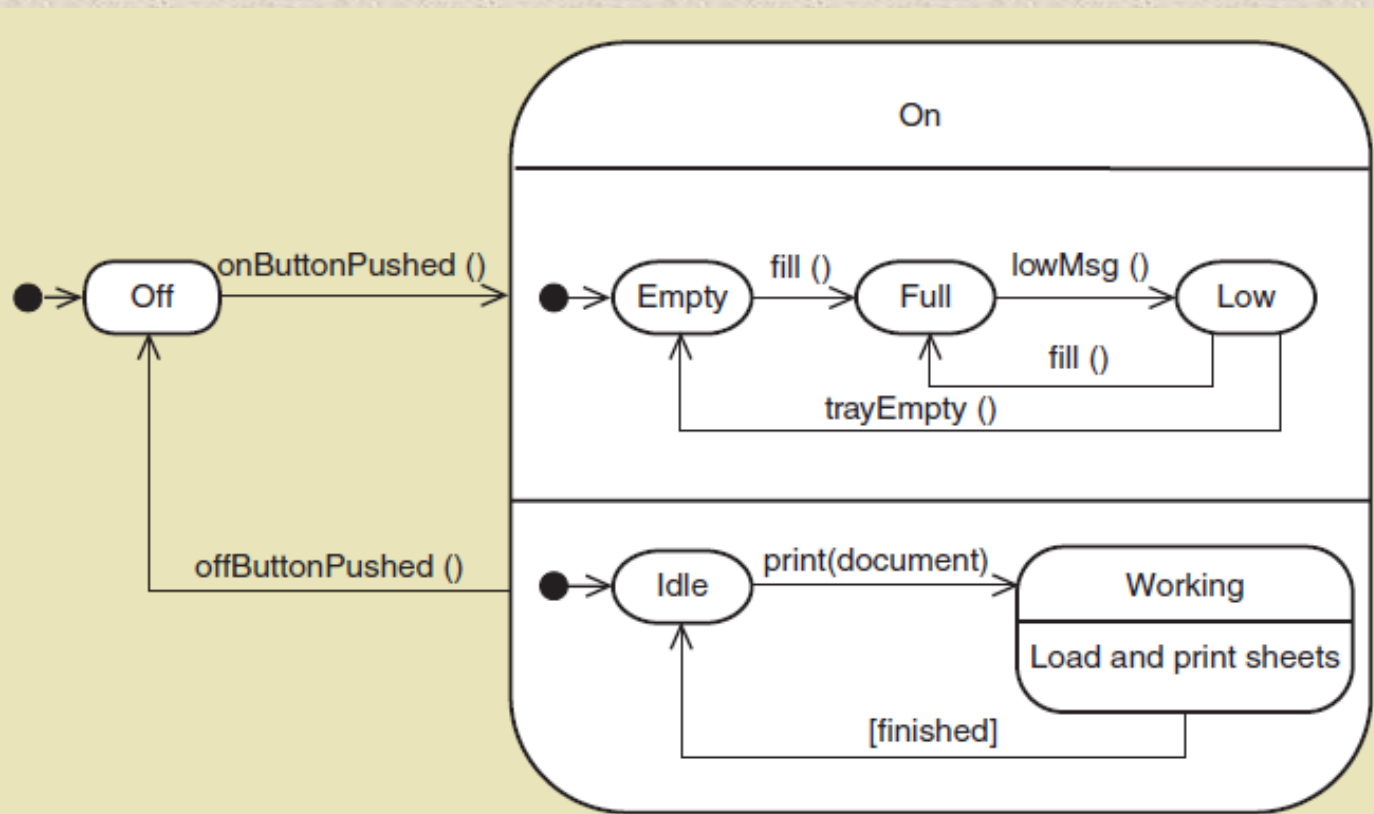


Example: Sequential Nested States for ATM System

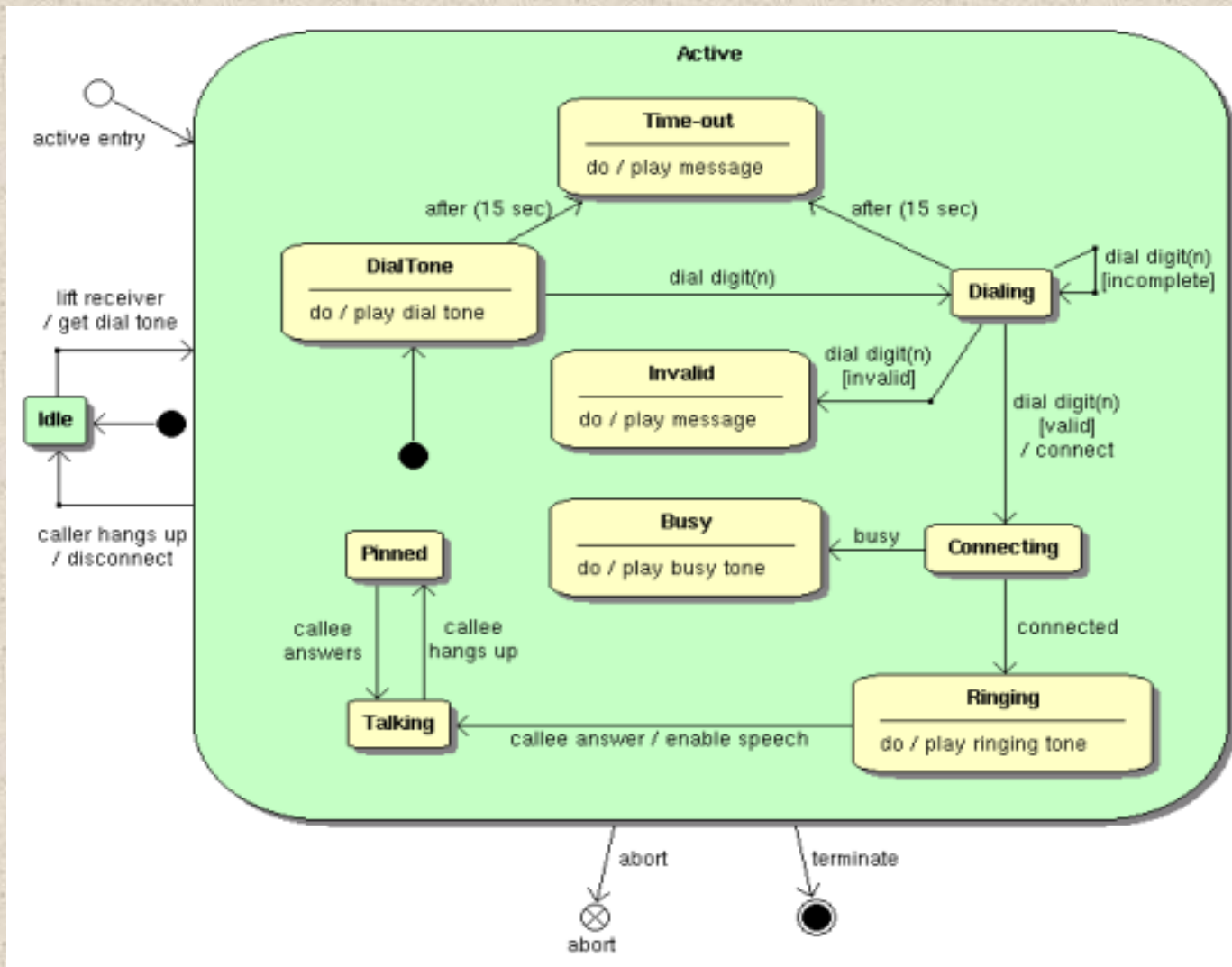


Concurrent composite states (e.g. for printer)

- The concurrent composite states let you specify two or more state machines that execute in parallel in the context of the enclosing object.



State diagram for phone line



Steps for Developing State Machine Diagram

1. Review the class diagram and select classes that might require state machine diagrams
2. For each class, make a list of status conditions (states) you can identify
3. Begin building diagram fragments by identifying transitions that cause an object to leave the identified state
4. Sequence these states in the correct order and aggregate combinations into larger fragments
5. Review paths and look for independent, concurrent paths

Steps for Developing State Machine Diagram (continued)

6. Look for additional transitions and test both directions
7. Expand each transition with appropriate message event, guard condition, and action expression
8. Review and test the state machine diagram for the class
 - Make sure state are really state for the object in the class
 - Follow the life cycle of an object coming into existence and being deleted
 - Be sure the diagram covers all exception condition
 - Look again for concurrent paths and composite states

RMO Class Diagram (Ref:Satzinger)

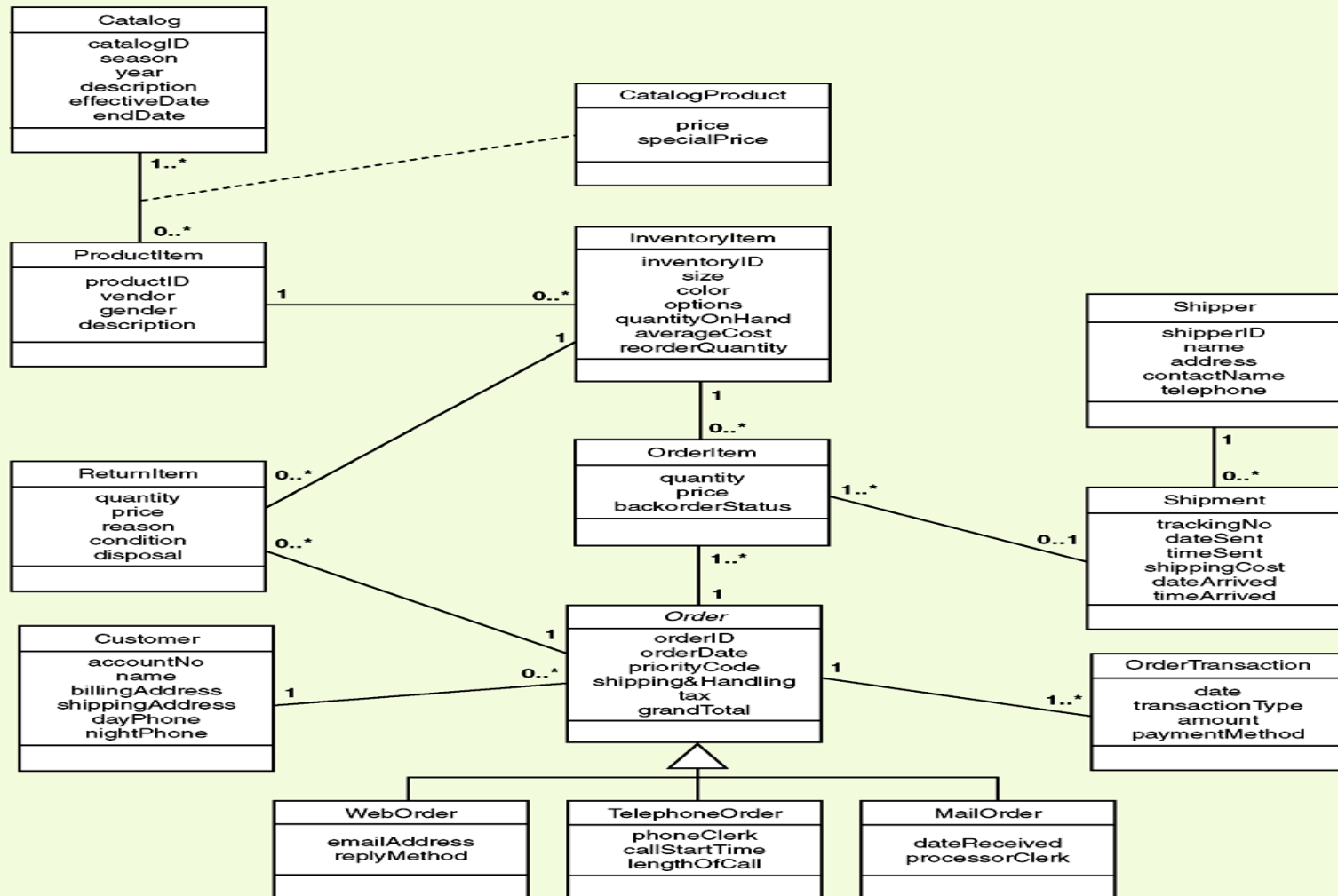
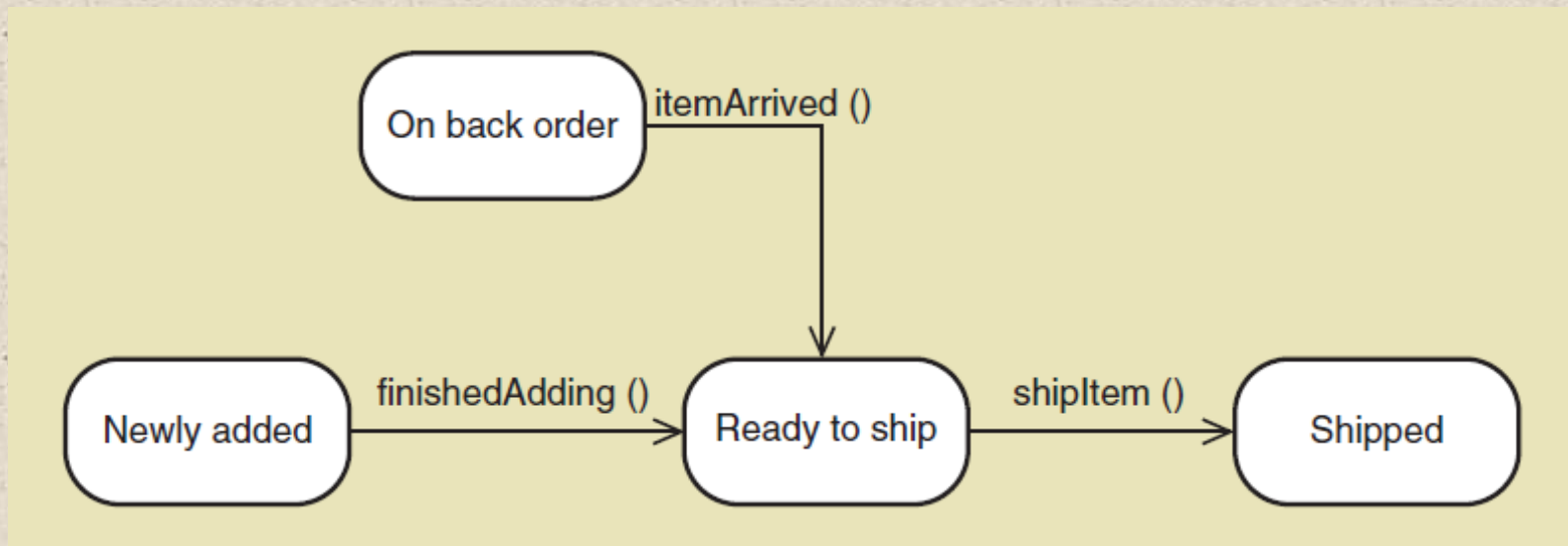


FIGURE 5-35
Rocky Mountain Outfitters class diagram.

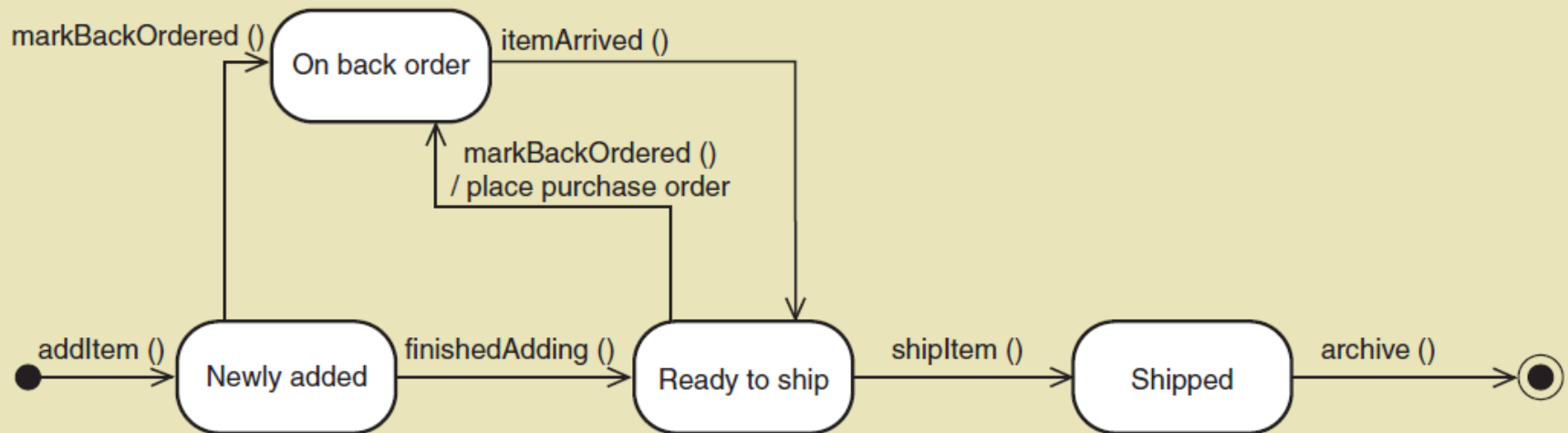
RMO Domain Class States for OrderItem Object

State	Transition causing exit from state
<i>Newly added</i>	finishedAdding
<i>Ready to ship</i>	shipItem
<i>On back order</i>	itemArrived
<i>Shipped</i>	No exit transition defined



Final State Machine Diagram for OrderItem Object

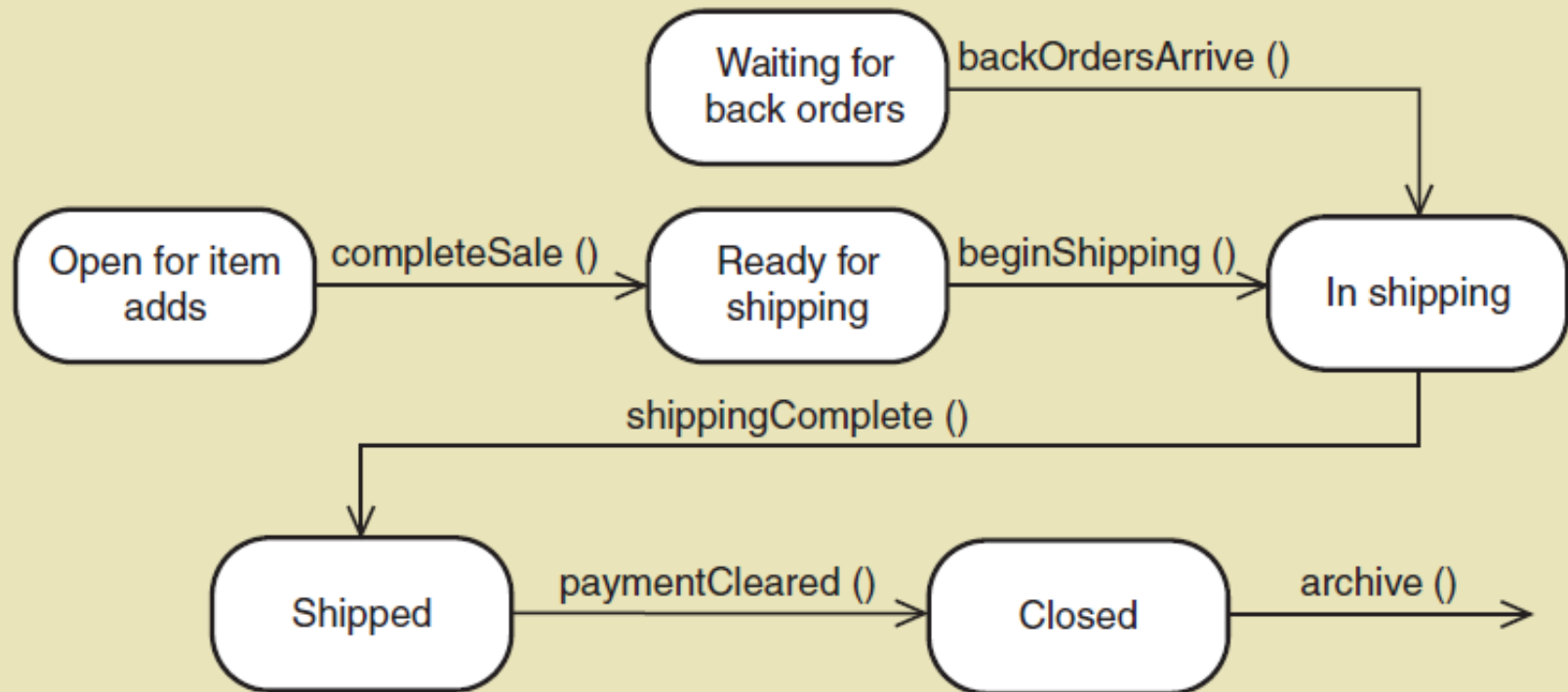
- addItem() and archive() transitions added
- markBackOrdered() transition added



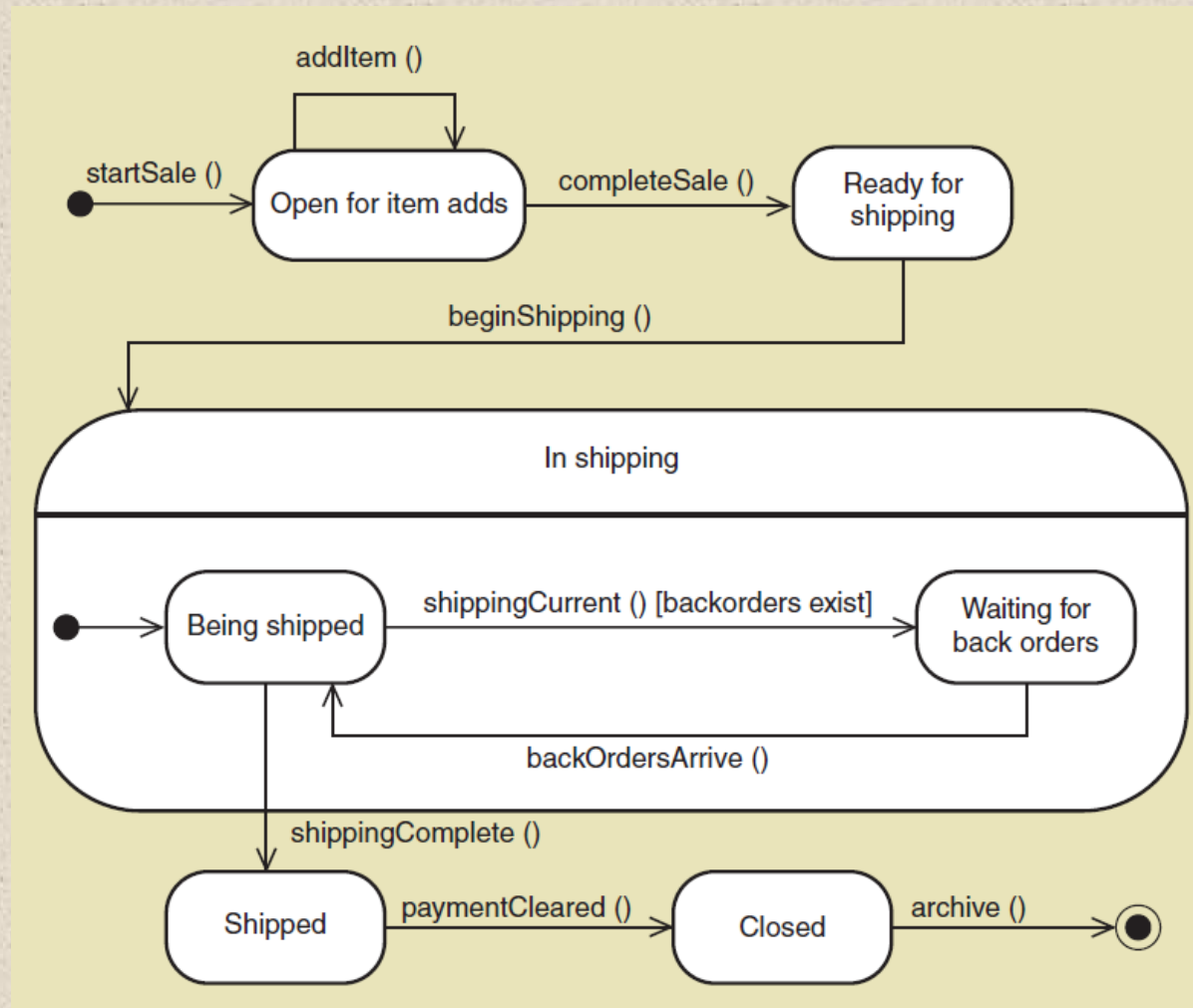
RMO Domain Class States for Order Object

State	Exit transition
<i>Open for item adds</i>	completeSale
<i>Ready for shipping</i>	beginShipping
<i>In shipping</i>	shippingComplete
<i>Waiting for back orders</i>	backOrdersArrive
<i>Shipped</i>	paymentCleared
<i>Closed</i>	archive

Initial State Machine Diagram for RMO Order Object



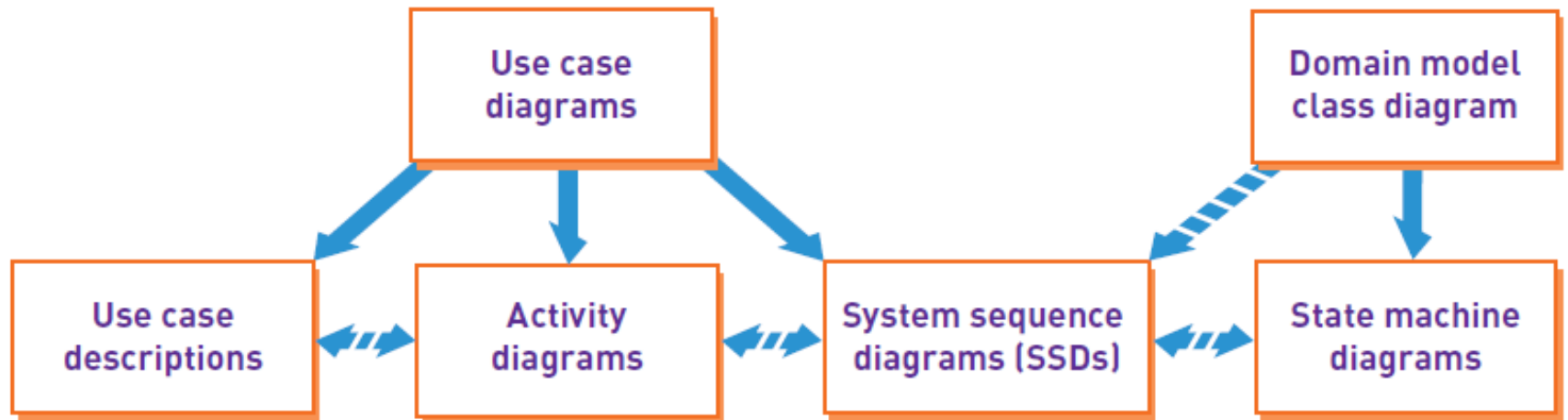
Final State Machine Diagram for Order Object



Extending and Integrating Requirements Models

- Use cases
 - Use case diagram
 - Use case description
 - Activity diagram
 - System sequence diagram (SSD)
- Domain Classes
 - Domain model class diagram
 - State machine diagram

Integrating Requirements Models



Questions??

- Define state chart diagram? What are the elements used in it?
- Draw state-chart diagram for ATM System.
- Define state, event, actions, sequential state and composite state.
- Consider a scenario for Library Management System. Draw class diagram and state diagram for relevant classes
- Draw state machine diagram for a printer. [D-15]

Questions??

- Consider the Hospital Management System application with following requirements and draw state-chart, component and deployment diagrams [D-15]:
 - System should handle the in-patient, out-patient information through receptionist.
 - Doctors are allowed to view the patient history and give their prescription.
 - There should be a information system to provide the required system.